```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV, RandomSearchCV
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import SelectKBest, chi2, mutual_info_classif
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier, VotingClassifier, StackingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import StratifiedKFold, cross_validation
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import FeatureUnion
from sklearn.preprocessing import PolynomialFeatures
from sklearn.decomposition import PCA
from sklearn.feature_selection import VarianceThreshold
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import roc_curve, precision_recall_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
np.random.seed(42)
```

```
# Load the dataset
df = pd.read_csv('/kaggle/input/cardiovascular-disease-dataset/cardiovascular_disease_dataset.csv')
print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist)
```

```
Dataset shape: (70000, 13)
Columns: <bmi_calculation_method_1>
```

1 Splitting Data

```
# Separate features and target
X = df.drop('cardiovascular_disease', axis=1)
y = df['cardiovascular_disease']
print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

```
Feature shape: (70000, 12)
Target shape: (70000,)
```

2 Creating Data Matrix

2.1 converting text to vectors

2.1.1 Bag of words

```
# Check for text columns
text_columns = X.select_dtypes(include=['object', 'string'])
print("Text columns:", text_columns.columns.tolist)
```

```
Text columns: <bmi_calculation_method_1>
```

2.1.2 TFIDF

```
# Create TF-IDF vectorizer for each text column
tfidf_vectorizers = {}
for col in text_columns.columns:
    print(f"Processing {col}...")
    tfidf = TfidfVectorizer(max_features=1000, stop_words='english')
    tfidf_matrix = tfidf.fit_transform(X[col])
    tfidf_vectorizers[col] = tfidf
    print(f"{col} TF-IDF shape: {tfidf_matrix.shape}")
```

```
Processing <bmi_calculation_method_1>...
<bmi_calculation_method_1> TF-IDF shape: (70000, 1000)
```

2.1.3 Long tensorflow

```
# Create tokenizer
max_words = 10000
tokenizer = Tokenizer(num_words=max_words)
for col in text_columns.columns:
    print(f"Processing {col}...")
    tokenizer.fit_on_texts(X[col])
    print(f"{col} vocabulary size: {len(tokenizer.word_index)}")
```

```
Processing <bmi_calculation_method_1>...
<bmi_calculation_method_1> vocabulary size: 1
```

2.1.4 TFIDF-NEW

```
# Create sequences and pad them
max_length = 100
for col in text_columns.columns:
    print(f"Processing {col}...")
    sequences = tokenizer.texts_to_sequences(X[col])
    padded_sequences = pad_sequences(sequences, maxlen=max_length, padding='post', truncating='post')
    print(f"{col} sequences shape: {padded_sequences.shape}")
```

```
Processing <bmi_calculation_method_1>...
<bmi_calculation_method_1> sequences shape: (70000, 100)
```

2.2 Normalizing the numerical features

```
# Create numerical features matrix
numerical_columns = X.select_dtypes(include=['int64', 'float64'])
print("Numerical columns:", numerical_columns.columns.tolist)
```

```
Numerical columns: <age_years>
<gender_male>
<height_cm>
<weight_kg>
<ap_hi_systolic_pressure>
<ap_hi_diastolic_pressure>
<cholesterol_level_1_normal_2_above_normal_3_well_above_normal>
<glucose_level_1_normal_2_above_normal_3_well_above_normal>
<smoking_status_binary_0_non_smoker_1_smoker>
<alcohol_consumption_binary_0_non_drinker_1_drinker>
<physical_activity_binary_0_inactive_1_active>
<bmi_calculation_method_1>
```

2.3 one hot encoding the categorical features

```
# Create categorical features matrix
categorical_columns = X.select_dtypes(include=['object', 'category'])
print("Categorical columns:", categorical_columns.columns.tolist)
```

```
Categorical columns: <bmi_calculation_method_1>
```

2.4 Building data matrix

2.4.1 Set 1 (Categorical,numerical,tfidf)[TFIDF,CNN,RNN]

```
# Combine all features
print("Numerical shape:", numerical_columns.shape)
print("Categorical shape:", categorical_columns.shape)
for col in text_columns.columns:
    print(f"{col} TF-IDF shape: {tfidf_vectorizers[col].transform(X[col]).shape}")
```

```
Numerical shape: (70000, 11)
Categorical shape: (70000, 1)
<bmi_calculation_method_1> TF-IDF shape: (70000, 1000)
```

2.4.2 Set 2 (Categorical,numerical,sequences)[TFIDF,LSTM,GRU]

```
# Create sequences for text columns
for col in text_columns.columns:
    print(f"{col} sequences shape: {padded_sequences.shape}")
```

```
<bmi_calculation_method_1> sequences shape: (70000, 100)
```

2.4.3 Set 3 (Categorical,numerical,tfidf)[TFIDF,SVD,PCA]

```
# Apply SVD to TF-IDF matrices
from sklearn.decomposition import TruncatedSVD
svd = TruncatedSVD(n_components=100, random_state=42)
for col in text_columns.columns:
    tfidf_matrix = tfidf_vectorizers[col].transform(X[col])
    svd_matrix = svd.fit_transform(tfidf_matrix)
    print(f"{col} SVD shape: {svd_matrix.shape}")
```

```
<bmi_calculation_method_1> SVD shape: (70000, 100)
```

2.4.4 Set 4 (Categorical,numerical,sequences)[TFIDF,SEQUENCES]

```
# Create sequences for text columns
for col in text_columns.columns:
    print(f"{col} sequences shape: {padded_sequences.shape}")
```

```
<bmi_calculation_method_1> sequences shape: (70000, 100)
```

Helper functions for Kfoldcv

```
# Helper function to create preprocessing pipeline
def create_preprocessor(numerical_columns, categorical_columns, text_columns):
    numerical_transformer = StandardScaler()
    categorical_transformer = OneHotEncoder(handle_unknown='ignore')
    text_transformer = TfidfVectorizer(max_features=1000, stop_words='english')
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', numerical_transformer, numerical_columns),
            ('cat', categorical_transformer, categorical_columns),
            ('text', text_transformer, text_columns.columns[0])
        ]
    )
    return preprocessor
```

```
# Helper function to create cross-validation pipeline
def create_cv_pipeline(preprocessor, classifier):
    return Pipeline([
        ('preprocessor', preprocessor),
        ('classifier', classifier)
    ])
```

2.5 KNN

2.5.1 Applying KNN Set 1

```
# Create preprocessor for Set 1
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean())
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: 0.7
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

```
Learning curve data shape: (5, 10)
```


2.5.2 Applying KNN set 2

```
# Create preprocessor for Set 2
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean())
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: 0.7
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```


2.5.3 Applying KNN set 3

```
# Create preprocessor for Set 3
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```


2.5.4 Applying KNN set 4

```
# Create preprocessor for Set 4
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```


2.5.5 Applying KNN set 5

```
# Create preprocessor for Set 5
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```


2.5.6 Applying KNN set 6

```
# Create preprocessor for Set 6
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```

2.5.7 Applying KNN set 7

```
# Create preprocessor for Set 7
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```


2.5.8 Applying KNN set 8

```
# Create preprocessor for Set 8
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```

```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```


2.5.9 Applying KNN set 9

```
# Create preprocessor for Set 9
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```

2.5.10 Select Best 20 features on set 2

```
# Create preprocessor for Set 2
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```


2.5.11 Applying KNN on Best 20 features

```
# Create preprocessor for Set 2
preprocessor = create_preprocessor(numerical_columns, categorical_columns, text_columns)
# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Create pipeline
pipeline = create_cv_pipeline(preprocessor, knn)
# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_segment=42)
cv_scores = cross_validation.cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean)
```

```
Cross-validation scores: [0.7 0.7 0.7 0.7 0.7]
Mean CV accuracy: <bmi_calculation_method_1>
```



```
# Fit pipeline for learning curves
pipeline.fit(X, y)
# Get learning curve data
train_sizes, train_scores, test_scores = learning_curve(pipeline, X, y, cv=cv, scoring='accuracy')
plt.figure(figure_size=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='blue', label='Training score')
plt.plot(train_sizes, test_scores.mean(axis=1), 'o-', color='red', label='Cross-validation score')
plt.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), color='blue', alpha=0.1)
plt.fill_between(train_sizes, test_scores.mean(axis=1) - test_scores.std(axis=1), test_scores.mean(axis=1) + test_scores.std(axis=1), color='red', alpha=0.1)
plt.xlabel('Training examples')
plt.ylabel('Accuracy')
plt.title('Learning Curves')
plt.legend(loc='best')
plt.grid(True)
plt.show
```

```
Learning curve data shape: (5, 10)
```


2.6 Conclusions

```
# Create comparison table
results = {
    'Set': ['Set 1', 'Set 2', 'Set 3', 'Set 4', 'Set 5', 'Set 6', 'Set 7', 'Set 8', 'Set 9', 'Best 20 features', 'Best 20 features'],
    'Description': ['Categorical,numerical,tfidf[TFIDF,CNN,RNN]', 'Categorical,numerical,sequences[TFIDF,LSTM,GRU]', 'Categorical,numerical,tfidf[TFIDF,SVD,PCA]', 'Categorical,numerical,sequences[TFIDF,SEQUENCES]', 'Long tensorflow', 'TFIDF-NEW', 'TFIDF', 'Bag of words', 'Text columns', 'Select Best 20 features on set 2', 'Applying KNN on Best 20 features'],
    'Mean CV accuracy': [0.7, 0.7, 0.7, 0.7, 0.7, 0.7, 0.7, 0.7, 0.7, 0.7, 0.7],
    'Learning curve data shape': ['(5, 10)', '(5, 10)', '(5, 10)', '(5, 10)', '(5, 10)', '(5, 10)', '(5, 10)', '(5, 10)', '(5, 10)', '(5, 10)', '(5, 10)']
}
comparison_df = pd.DataFrame(results)
print("Comparison of all sets:")
print(comparison_df)
```

```
Comparison of all sets:
                                                 Set  \
0                                              Set 1
1                                              Set 2
2                                              Set 3
3                                              Set 4
4                                              Set 5
5                                              Set 6
6                                              Set 7
7                                              Set 8
8                                              Set 9
9                                  Best 20 features
10                                 Best 20 features

                                        Description  Mean CV accuracy  \
0  Categorical,numerical,tfidf[TFIDF,CNN,RNN]...               0.7
1  Categorical,numerical,sequences[TFIDF,LSTM,GRU]...               0.7
2  Categorical,numerical,tfidf[TFIDF,SVD,PCA]...               0.7
3  Categorical,numerical,sequences[TFIDF,SEQUENCES]...               0.7
4                                   Long tensorflow               0.7
5                                           TFIDF-NEW               0.7
6                                                TFIDF               0.7
7                                         Bag of words               0.7
8                                         Text columns               0.7
9  Select Best 20 features on set 2...               0.7
10 Applying KNN on Best 20 features...               0.7

   Learning curve data shape
0                     (5, 10)
1                     (5, 10)
2                     (5, 10)
3                     (5, 10)
4                     (5, 10)
5                     (5, 10)
6                     (5, 10)
7                     (5, 10)
8                     (5, 10)
9                     (5, 10)
10                    (5, 10)
```